*This document constitutes a reference description form to facilitate and set a standard for information exchange between stakeholders on the definition of a **control function (CF)**. The CF is a description of an embedded control system (or an aspect thereof) which, together with the inherent physical properties of the system itself, defines the behavior of the control system and its response to dynamic input. Control functions can be seen as a further detailing, or more formal description, of the mechanisms governing the behavior of individual actors in a use case.*

Control functions may take the form of mathematical functions, such as transfer functions or descriptions of a stochastic process, or they may be best described by an algorithm. These different forms require different description options. As a result, some of the sections of the document are optional because their content does not apply to all descriptions. This is indicated at the beginning of each section. Optional sections should be left blank, rather than being deleted entirely, in order to preserve the consistency of section numbers. Sections marked as mandatory must be filled in every case.

NOTE: This document has been carefully structured in a way that allows to extract the contained information in an automated way and to translate it to other machine-readable formats (parsing and conversion by script). Therefore, when editing this document, please keep the following in mind:

- *Only information (text and figures) provided in the cells of the tables defined below will be extracted.*
- *The first cell of each table contains a specific title that identifies its purpose ("Document History", "Functional Description", etc.). Do NOT change these titles, even if you replicate a table (as in the case for the control function inputs).*
- *When extending tables (e.g., add more instances to an element description), insert new row with the same layout. Especially do NOT split or merge cells in a table's row.*
- *You may add additional (sub-)headings to structure the document to accommodate your needs.*
- *Names and identifiers should be machine-readable, i.e., no spaces, no special symbols, only basic ASCII letters and numbers.*

1. Document History

Please keep track of significant changes to this document. Consider using the following versioning scheme to indicate the stage of the CF: 0.x – concept stage, 1.x – test design stage, 2.x – post test / documentation stage, 3.x and higher – revisiting the test.

Document History			
Date	Version	Description	Author

2. Control Function Identification

Please specify a name for this CF together with an ID to identify it.

NOTE: Please use a machine-readable ID.

Control Function Identification	
Control Function ID	Name

3. Functional Description

The purpose here is to provide a brief introduction to the problem solved by the function or the algorithm. The technical background should be outlined by giving an overview of the function or algorithm. It will be important in the early introduction of the document to clearly inform the reader on the intent of the function or algorithm as well as the problem the function or algorithm is addressing. The author should keep in mind that this document is not a technical paper and should be written in such a way that it can be understood by an audience with limited expertise in specific simulation domains.

The statement of the problem should include a clear statement outlining the problem and why the problem is important and needs to be addressed. It should contain a sketch of the problem/situation and, if known and/or relevant, the background history that led to the development of the function or algorithm, including why the problem was investigated and how it relates to earlier work that has been done in the field. It should show that the author thoroughly understands the problem.

Functional Description

4. Terminology (optional)

The role of terminology is important, and it defines concepts used in the scientific document. The definitions should be made precise, concise, and unambiguous. Each term should be used in one and only one way throughout the document.

Terminology	

5. Methodology

This section is used to provide the methodology or theory of the function or algorithm in detail. Relevant research results should be referenced. The description should not be as detailed as a peer-review journal paper. However, all information necessary for understanding specifics of the function or algorithm, especially for translating it into software, must be included. Required equations and computations should be described and explained term by term and be closely related to the detailed description of the function or algorithm later in the document.

In case of well-known or standard functions or algorithms, a simple reference to the theory (e.g., a link to the Wikipedia page for "PID controller") is sufficient.

Methodology

6. Limitations

This section describes prerequisites for the function or algorithm. These include limitations caused by input data, special types of data required, limitations inherent to the function or algorithm, situations where the function or algorithm cannot be applied, potential implementation difficulties, and any other problems known to be associated with the function or algorithm. This section should also describe both the implementation and operational impacts of these limitations.

Limitations

7. Inputs

Please replicate the table below for each control input. This table describes data inputs to the documented function or algorithm. This includes static input data such as configuration or parametrization as well as dynamic input data such as measurement data or events. Each input data element should specify the data type (e.g., floating point, integer, character string), units (e.g., degrees, percentages), valid ranges (e.g., 0 to infinity; -1.0 to +1.0 inclusive), the expected update rate of the data (e.g., static; 10Hz; daily) and a short description of the information the variable contains (e.g., "width of the radar beam in degrees …").

All data element names should be self-descriptive (i.e., the name should describe what the data element contains). Widely recognized symbols from textbooks and professional literature are acceptable.

NOTE: Please use machine-readable names.

Control Function Input	
Name	
Type (according to SC description)	
Unit	
Range	
Expected update rate	
Description	

8. Outputs

Please replicate the table below for each control output. This table describes data output produced by the documented function or algorithm, insofar as the output is relevant to the interaction between the function or algorithm and the rest if the system. This covers mostly dynamic output data such as

setpoints or events. Each output data element should describe the format of the output data in sufficient detail for downstream algorithms to be easily interfaced. This should as a minimum include the data type (e.g., floating point, integer, character string), units (e.g., degrees, percentages), valid ranges (e.g., 0 to infinity; -1.0 to +1.0 inclusive), the expected update rate of the data (e.g., static; 10Hz; daily) and a short description of the information the variable contains (e.g., "power setpoint in kW").

All data element names should be self-descriptive (i.e., the name should describe what the data element contains). Widely recognized symbols from textbooks and professional literature are acceptable.

NOTE: Please use machine-readable names.

Control Function Output	
Name	
Type (according to SC description)	
Unit	
Range	
Expected update rate	
Description	

9. Use Cases

Use Cases are a way to express functional requirements. They bridge the gap between user needs and system functionality by directly stating the user's intention and system response for each step in a particular interaction. No single Use Case specifies the entire requirements of the system. A Use Case itself is an interaction that a User or another System has with the system that is being designed to achieve a goal. Different scenarios within a Use Case show how the goal succeeds or fails. A success scenario is one in which the goal is achieved; a failure scenario is one where the goal is not achieved. Because the goals summarize the intention of the various uses of the system, the users can see how they are supposed to use the system. Users can also spot when the system does not support all of their goals without having to wait for the first prototype or having to wait for the system to be developed.

This is a summary of how to write Use Cases:

- *The name of the use case should start with a strong verb.*
- *A Use Case is a set of scenarios. A scenario is a list of steps.*
- *Each step should state what the user does and/or how the system responds.*
- *The steps must not mention how the system does something.*
- *Each step needs to be analyzed in detail before it becomes code.*
- *Keep It Simple: use the simplest format you need.*
- *Keep track of different versions.*
- *Writing use cases is a team sport.*
- *Focus on a particular user (give them a name).*

Don't get bogged down in all the special ways it can go wrong until you've finished the main success story.

Use Case Example	*<Use Case name>*
Date created	
Actor	
Description	
Preconditions	
Postconditions	
Priority	
Frequency of use	
Normal course	
Alternative course	
Exceptions	
Assumptions	
Notes and issues	

10. Diagrams (data flow diagrams, sequence diagrams, logic diagrams, state diagrams, control hierarchy, etc.)

This section will contain diagrams which capture the data flow occurring between a function or algorithm and the rest of the system. These diagrams should reveal relationships among and between the various components in a program or system.

In case of distributed controllers, i.e., multiple processes communicating to achieve an overall objective, sequence diagrams of the interaction between the individual processes must be provided here, in addition to the detailed descriptions of the individual processes in subsequent sections.

Where applicable, this section should also contain Logic Diagrams and/or State Diagrams. Logic Diagrams represent logical concepts and State Diagrams represent the behavior of a system. State diagrams describe all the possible states of an object as events occur. Each diagram usually represents objects of a single class and tracks the different states of its objects through the system.

NOTE: Please put each drawing in a separate table cell. Each table cell with a drawing should be preceded by a table cell containing the drawing's title.

Diagrams
<Diagram title>

11. Algorithmic Functions (pseudocode)

This section is optional; however, at least one of the three detailed description sections (Deterministic Functions, Stochastic Functions, Algorithmic Functions) must be filled in.

This section will contain the description of algorithms in a step-by-step fashion, preferably using pseudocode if applicable.

The term "pseudocode" is used here to describe an English-like language that articulates the steps carried out in an algorithm. It allows the author to focus on the logic of the algorithm without being distracted by details of any given programming language's syntax. For the sake of completeness and consistency, pseudocode should follow a general syntax convention (see the appendix). Pseudocode can be augmented with natural language where convenient.

In case of distributed controllers, i.e., multiple processes communicating to achieve an overall objective, sequence diagrams of the interaction between the individual processes must be provided in the "Diagrams" section, in addition to the detailed descriptions of each of the individual processes here.

NOTE: Please put each algorithm in a separate table cell. Each table cell with an algorithm should be preceded by a table cell containing the algorithm's title.

Algorithms
<Algorithm title>

12. Deterministic Functions

This section is optional; however, at least one of the three detailed description sections (Deterministic Functions, Stochastic Functions, Algorithmic Functions) must be filled in.

This section will contain a mathematical description of the deterministic (as opposed to stochastic) part of a function. Each mathematical formula should be followed by a brief explanation of the symbology.

The section can also be used to document the mathematical background of an algorithm for which pseudocode is provided in the "Algorithms" section below. In this case, the description should also cover the method used to realize the implementation in the Algorithms section.

Deterministic Functions

13. Stochastic Functions

This section is optional; however, at least one of the three detailed description sections (Deterministic Functions, Stochastic Functions, Algorithmic Functions) must be filled in.

This section will contain a mathematical description of the stochastic (as opposed to deterministic) part of a function. Each mathematical formula should be followed by a brief explanation of the symbology.

The section can also be used to document the mathematical background of an algorithm for which pseudocode is provided in the "Algorithms" section below. In this case, the description should also cover the method used to realize the implementation in the Algorithms section.

Stochastic Functions

14. Deployment (optional)

This section is optional. It should be filled in if the described function or algorithm is deployed in a concrete setup (e.g., as part of a lab experiment).

This section provides additional information regarding the deployment of the control function in a hardware and/or software setup.

Deployment (optional)

15. References

References